When Small Action Errors Matter: Closed-Loop Analysis of Inference Acceleration for VLA Policies

patrick.zhang
patrick.zhang5233@gmail.com

Abstract

Inference acceleration for vision-language-action (VLA) policies is usually framed as a deployment problem: a full-precision policy is quantized, compiled, fused, or otherwise re-executed more cheaply, and the result is evaluated by output drift, task accuracy, latency, or memory. For VLA policies, this framing is incomplete. A VLA model is not only a predictor; it is a closed-loop controller. We argue that post-training inference acceleration should be evaluated as a closed-loop policy perturbation. Quantization, graph compilation, mixed precision, and kernel replacement can all change the implemented policy function slightly, which can change the state distribution induced by robot-environment interaction. Small action errors can therefore be amplified by contact dynamics, receding-horizon feedback, and thresholded success conditions.

We study this effect through two acceleration tracks for a GR00T N1.5 policy on LIBERO-10. Track A is a reproduction-oriented analysis of selective W4A8 fake quantization. Offline teacher-student action drift shows that attention balancing methods can improve mean action error, but also worsen a non-trivial subset of held-out observations. In closed-loop simulation over 150 matched task-initialization pairs, selective W4A8 variants remain in the FP16 behavioral range under our evaluation protocol, but their gains are not monotonic: compensation methods repair some failed rollouts while introducing new failures elsewhere. The highest point-estimate single compensation mode, output head balancing (OHB), reaches 116/150 successes compared with 113/150 for uncompensated W4A8 and 108/150 for FP16, yet it still regresses several task slices. Track B studies prototype acceleration boundaries using `torch.compile` and eager islands in the action head. Controlled perturbation and acceleration-boundary experiments show that closed-loop sensitivity is anisotropic across action dimensions, rollout phases, and model-layer boundaries. A small proxy-guided mixed-precision probe supports the resulting design rule: protecting closed-loop-sensitive action-head blocks can recover some speed-only regressions while preserving part of the latency gain, but it cannot eliminate trajectory redistribution. We do not claim statistically significant closed-loop superiority from these small aggregate gaps. Our study focuses on behavior under fake quantization and prototype acceleration boundaries rather than final packed-kernel deployment efficiency. The results suggest that VLA inference acceleration should be evaluated as policy perturbation, not merely as numerical approximation or systems optimization.

1 Introduction

Large vision-language-action models are increasingly used as generalist robot policies. They map visual observations and language instructions into action chunks, often through diffusion-style denoising or transformer action heads. As these models become larger, inference acceleration is a natural route toward cheaper deployment. This includes quantization, graph compilation, kernel replacement, CUDA graph replay, caching, and mixed-precision execution. However, common acceleration evaluations focus on local approximation quality or systems throughput: weight error, activation error, output MSE, single-step action drift, latency, memory, or energy. These metrics are useful, but they do not fully capture what matters for embodied control.

In a closed-loop policy, an action error at time t changes the next observation. The next action is then produced on a state that may not have been visited by the original full-precision policy. This feedback loop creates a mismatch between local teacher-student error and final task success. The issue is especially sharp in manipulation, where small geometric deviations can change contact timing, grasp stability, object pose, or whether a state satisfies a binary success predicate.

This paper studies the following question:

When a VLA policy is accelerated after training, how should we interpret small action errors in terms of closed-loop task success?

Our answer is that post-training acceleration should be treated as a policy perturbation. It maps a reference policy into a constrained implementation, such as a low-precision model, a compiled graph, or a mixed eager/compiled execution plan, thereby changing both the conditional action distribution and the induced state distribution. The relevant empirical object is not only local action drift on fixed observations, but also how the perturbed implementation redistributes success and failure across matched closed-loop rollouts.

We make five observations:

1. Selective W4A8 quantization can be behaviorally robust even when it measurably changes action outputs.
2. Attention-balancing corrections can improve mean offline drift while still worsening individual observations and task slices.
3. Paired closed-loop rollout flips expose repair/regression structure that aggregate success rates hide.
4. Closed-loop sensitivity is anisotropic: the same perturbation norm can have different effects depending on action channel, rollout phase, and layer/boundary location.
5. Sensitivity-guided protection can improve the speed–robustness trade-off, but does not make accelerated execution behaviorally identical to the FP16 baseline.

Our contributions are:

1. We formulate post-training inference acceleration for VLA policies as a closed-loop policy perturbation problem, where the main object is the induced state distribution rather than only local action approximation or raw latency.
2. We derive local perturbation claims showing that deployment-induced action error matters through sensitivity-weighted propagation and terminal success-margin crossing, and formulate a closed-loop criterion for future correction and protection methods.
3. We provide a reproduction-oriented GR00T/LIBERO case study showing that selective W4A8 fake quantization can remain in the FP16 behavioral range while changing which task-initialization pairs succeed.
4. We show that offline mean drift and aggregate success rate can both obscure important structure, and advocate paired repair/regression analysis as a compact diagnostic for VLA acceleration.
5. We empirically map closed-loop sensitivity across action dimensions, trajectory phases, and action-head layer boundaries, showing that safety and speed cannot be predicted from perturbation norm or latency alone.
6. We test a small sensitivity-proxy-guided mixed-precision execution policy, showing that closed-loop proxies can reduce speed-only regressions while still requiring paired trace analysis to expose remaining failures.

From point solution to design-space map. Our goal is not to present a single operating point in the VLA acceleration design space. Instead, we use quantization and compilation probes to expose structure in the problem: which perturbations are absorbed, which are amplified by closed-loop dynamics, which rollouts are repaired or regressed, and which modules or states should be protected by future methods. These properties are useful beyond our specific implementation. They suggest how later work might choose calibration states, design residual action corrections, select mixed-precision modules, compile only insensitive boundaries, or trigger high-precision fallback around fragile trajectory regions.

Model-class interpretation. Our findings are not tied to a specific checkpoint-level failure pattern. They suggest a model-class behavior: for DiT-style VLA or world-action models, small acceleration-induced implementation perturbations are filtered through closed-loop sensitivity, task margins, and trajectory basins. Therefore, larger or newer GR00T-style variants, as well as related action-diffusion policies, should be evaluated by re-estimating their sensitivity maps rather than assuming uniform acceleration tolerance across dimensions, time, or layers.

Scope of claims. This paper should be read as a behavior-level reproduction study and diagnostic analysis, not as a new quantization algorithm, compiler, or deployment benchmark. We do not claim final low-precision latency, memory, or energy improvements because the quantization track uses fake quantization rather than packed low-precision kernels, and the compilation track is a prototype action-head acceleration probe. We also do not treat small aggregate success-rate differences as evidence of policy dominance. The main empirical claim is narrower: VLA acceleration can preserve aggregate behavior while redistributing which closed-loop rollouts succeed, so paired rollout analysis is necessary to interpret the effect of small action perturbations. The resulting design principle is that VLA acceleration should target sensitivity-weighted closed-loop perturbation rather than only average open-loop action error or raw serving latency.

Experimental roadmap. The experiments are intentionally staged. Track A studies quantization-induced perturbations: selective W4A8 fake quantization, ATM/OHB compensation, offline action drift, and 150 matched closed-loop rollouts. Track B studies execution-induced perturbations: `torch.compile`, eager islands, warm serving latency, and paired rollout flips. Between them, bridge experiments map which perturbations are sensitive across action dimensions, rollout windows, and layer/boundary locations. The `torch.compile` experiments should not be read as a quantization method. They are an acceleration track used to ask whether a speed-oriented execution boundary can also be amplified by closed-loop dynamics, and whether sensitivity proxies can guide which boundaries to protect.

2 Related Work

Vision-language-action policies. Recent robot policies increasingly combine large-scale visual, language, and action modeling. RT-1 introduced a scalable transformer policy trained on diverse real-robot data [2]. RT-2 framed robotic actions as tokens inside a vision-language-action model and showed transfer from web-scale vision-language training to control [3]. OpenVLA made this direction more accessible by releasing a 7B-parameter open-source VLA trained on large robot demonstration mixtures, and also noted practical interest in efficient fine-tuning and quantized serving [7]. GR00T N1 uses a dual-system VLA architecture where a vision-language module conditions a diffusion transformer action module [10]. Diffusion Policy and related action-diffusion methods model robot behavior as conditional denoising, often with receding-horizon control [4]. Our work focuses on the evaluation problem that appears when such policies are compressed or accelerated after training.

Inference acceleration for closed-loop policies. Deployment acceleration can come from low-precision arithmetic, graph compilation, operator fusion, CUDA graph replay, caching, or custom kernels. In open-loop perception or language tasks, small numerical differences are usually judged by output agreement and throughput. In VLA policies, the accelerated implementation chooses actions that alter future observations. This makes acceleration a behavioral question as well as a systems question: a faster implementation is useful only if its closed-loop policy perturbation stays within acceptable task margins.

Post-training quantization. PTQ methods for large transformers often target memory and latency while preserving model quality. LLM.int8() handles transformer outliers through mixed-precision decomposition [5]; GPTQ uses approximate second-order information for one-shot low-bit weight quantization [6]; SmoothQuant migrates quantization difficulty from activations to weights through an equivalent smoothing transform [12]; and AWQ uses activation statistics to protect salient weight channels [8]. These methods demonstrate that transformer quantization can be accurate when calibration accounts for activation structure. However, most evaluations are open-loop language, vision-language, or recognition tasks. In VLA

policies, the quantized model selects actions that alter future inputs, so the calibration and evaluation target must include closed-loop effects.

Quantization for VLA models. QuantVLA is, to our knowledge, the most direct prior work on PTQ for VLA systems. It proposes a scale-calibrated PTQ framework with selective quantization, attention temperature matching, and output head balancing, and reports memory and latency gains together with LIBERO success-rate improvements [13]. Our study is not a deployment-efficiency reproduction of those claims. Instead, we use a behavior-level fake-quantization reproduction to analyze why VLA quantization can improve some rollouts while regressing others, and why paired closed-loop evaluation is necessary even when average offline drift improves.

Sequential prediction and distribution shift. The gap between offline error and closed-loop behavior is a classic issue in imitation learning and sequential prediction. DAgger formalizes the problem that future observations depend on previous actions, so policies trained or evaluated only under a fixed data distribution can suffer from compounding errors [11]. VLA acceleration creates a related but distinct problem: the policy is already trained, but a post-training implementation change perturbs its action outputs and therefore perturbs the state distribution it induces. This makes paired rollout analysis a natural complement to offline teacher-student action drift.

Feedback control perspective. Classical feedback control emphasizes that closed-loop behavior is determined by the interaction between controller perturbations, plant dynamics, and feedback gain [1]. We use this lens to interpret inference acceleration not as a static approximation problem, but as a bounded policy perturbation whose effect depends on local closed-loop sensitivity and task margins.

3 Inference Acceleration as Policy Perturbation

Let $\pi_\theta(a \mid s, l)$ denote a reference FP16 VLA policy conditioned on state observation s and language instruction l. A post-training acceleration transform constructs an implemented policy

$$\varphi = \mathcal{A}(\theta; C, S), \qquad \pi_q = \pi_\varphi, \tag{1}$$

where $\mathcal{A}$ may represent quantization, graph compilation, mixed-precision partitioning, kernel replacement, or graph replay; C denotes calibration data when applicable; and S denotes systems choices such as compilation boundaries or shape buckets. We keep the shorthand π_q for the perturbed implementation because the first experimental track is quantization, but the derivation applies to any acceleration path that induces a small action perturbation. Even without gradient updates, this is not a neutral storage transform: it changes the implemented function.

Offline action drift measures a local difference such as

$$e(s) = \|\pi_q(s, l) - \pi_\theta(s, l)\| \tag{2}$$

on observations sampled from some dataset or teacher rollout distribution. Closed-loop performance depends on the state distributions induced by the two policies, d_{π_θ} and d_{π_q}.

For a local transition model,

$$s_{t+1} = f(s_t, a_t), \qquad a_t = \pi(s_t), \tag{3}$$

a first-order error expansion gives

$$\delta s_{t+1} \approx J_s \delta s_t + J_a \delta a_t, \qquad \delta a_t = \pi_q(s_t) - \pi_\theta(s_t). \tag{4}$$

Even if δa_t is small on the teacher state distribution, the accumulated δs_t can move the accelerated policy into states where the original local error estimate no longer applies.

This is why manipulation success is not a smooth function of single-step action MSE. A rollout can flip because of a small action perturbation if that perturbation changes whether the gripper reaches the object before closing, whether a placed object crosses a success-region boundary, whether an object contact

produces a different pose, or whether a receding-horizon action chunk enters a different trajectory basin. In this view, acceleration is closer to post-training policy editing than to passive systems optimization.

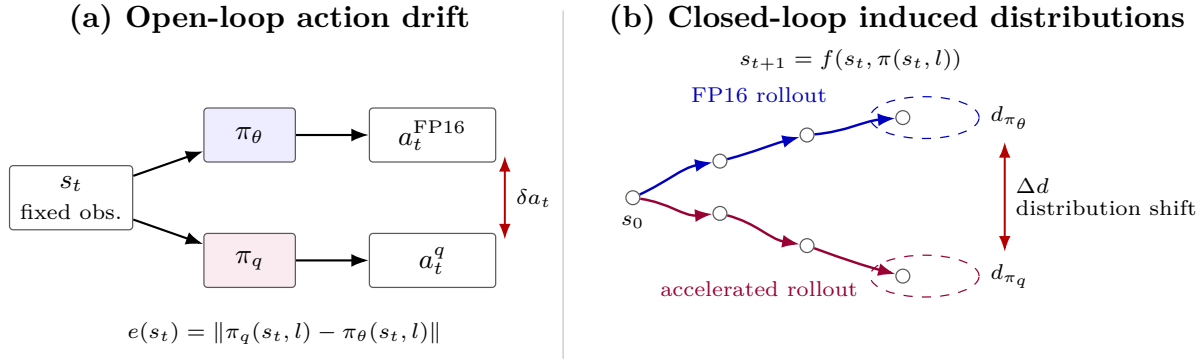

Figure 1: Inference acceleration as closed-loop policy perturbation. Offline action drift compares two actions on a fixed observation, while closed-loop execution lets each policy induce its own future observation distribution.

Claim 1: implementation error is filtered by closed-loop sensitivity. The first-order expression above can be expanded over a horizon. Let $\eta_t = \pi_q(s_t, l) - \pi_\theta(s_t, l)$ be the local acceleration-induced action perturbation on the nominal FP16 trajectory, let $A_t = \partial f/\partial s$, $B_t = \partial f/\partial a$, and $K_t = \partial \pi_\theta/\partial s$ be evaluated along that trajectory, and define the local closed-loop Jacobian

$$F_t = A_t + B_t K_t. \tag{5}$$

This claim is local and first-order: it assumes small perturbations around the FP16 trajectory, defines η_t on that nominal trajectory, and approximates the accelerated policy's local state Jacobian by the FP16 policy Jacobian, ignoring differences between $\partial \pi_q/\partial s$ and $\partial \pi_\theta/\partial s$ at this order. Under these assumptions, the terminal state perturbation satisfies

$$\delta s_T \approx \sum_{t=0}^{T-1} \left(F_{T-1} F_{T-2} \cdots F_{t+1}\right) B_t \eta_t, \tag{6}$$

with the empty product interpreted as the identity. Thus the effect of acceleration is not determined by $\|\eta_t\|$ alone, but by how its direction aligns with transition dynamics, policy feedback, and future closed-loop gain.

Claim 2: rollout flips are margin-crossing events. Let $h(s_T)$ be an implicit terminal success margin, with success when $h(s_T) > 0$. For a small terminal perturbation,

$$h(s_T^q) - h(s_T) \approx \nabla h(s_T)^\top \delta s_T. \tag{7}$$

A rollout can flip when the projected perturbation is comparable to the FP16 margin:

$$\left|\nabla h(s_T)^\top \delta s_T\right| \gtrsim \left|h(s_T)\right|. \tag{8}$$

More precisely, the outcome flips when the perturbation changes the sign of the margin:

$$\operatorname{sign}(h(s_T^q)) \neq \operatorname{sign}(h(s_T)). \tag{9}$$

If the FP16 rollout succeeds, $h(s_T) > 0$, an accelerated regression requires

$$\nabla h(s_T)^\top \delta s_T \lesssim -h(s_T). \tag{10}$$

If the FP16 rollout fails, $h(s_T) < 0$, an accelerated repair requires

$$\nabla h(s_T)^\top \delta s_T \gtrsim |h(s_T)|. \tag{11}$$

Thus Equation 8 should be read as a magnitude condition for either flip direction; the sign determines whether the perturbation causes a repair or a regression. This explains why two rollouts with similar single-step action drift can behave differently: high-margin trajectories absorb the perturbation, while low-margin trajectories near contact or placement boundaries can cross the success threshold.

Design principle: optimize sensitivity-weighted closed-loop perturbation. Equations 6 and 8 suggest that the relevant post-training acceleration objective for VLA policies is not the average open-loop action deviation

$$\min_{Q} \ \mathbb{E} \sum_{t} \|\eta_t(Q)\|^2, \tag{12}$$

but a sensitivity-weighted perturbation objective of the form

$$\min_{Q} \ \mathbb{E} \sum_{t} \eta_t(Q)^\top G_t \eta_t(Q), \tag{13}$$

where one local terminal-margin approximation is

$$G_t = B_t^\top \Phi_{t\to T}^\top \nabla h(s_T) \nabla h(s_T)^\top \Phi_{t\to T} B_t, \qquad \Phi_{t\to T} = F_{T-1} \cdots F_{t+1}. \tag{14}$$

Equation 13 is a separable local surrogate, not the exact squared full-horizon terminal-margin objective. To see this, define the scalar contribution

$$\alpha_t = \nabla h(s_T)^\top \Phi_{t\to T} B_t \eta_t = c_t^\top \eta_t, \qquad c_t = B_t^\top \Phi_{t\to T}^\top \nabla h(s_T).$$

The first-order terminal margin perturbation is $\Delta h \approx \sum_t \alpha_t$. Its exact squared approximation expands as

$$(\Delta h)^2 \approx \left(\sum_t \alpha_t \right)^2 = \sum_t \alpha_t^2 + 2 \sum_{i<j} \alpha_i \alpha_j.$$

The per-step term is $\sum_t \alpha_t^2 = \sum_t \eta_t^\top c_t c_t^\top \eta_t$, which gives Equation 13 with $G_t = c_t c_t^\top$. The omitted cross-time terms represent interactions between implementation errors at different rollout steps: they can amplify risk when their margin effects have the same sign, or partially cancel when their effects have opposite signs. We use the separable form as a tractable proxy, appropriate when cross-time error correlations are weak or when the objective is meant to rank locally sensitive perturbation directions rather than exactly optimize rollout-level margin loss. In practice G_t is difficult to estimate exactly, but the expression identifies the target: reduce implementation error in state-action directions that are amplified by feedback dynamics and projected onto small task margins. Layer-statistic corrections such as ATM/OHB and execution-boundary protection can be useful proxies, but they do not by themselves optimize this closed-loop criterion.

Claim 3: closed-loop sensitivity is anisotropic. The metric G_t also implies that not all perturbations of the same norm are equally important. For a unit action coordinate e_j, an equal-magnitude perturbation $\eta_t = \epsilon e_j$ has local squared terminal-margin risk

$$r_{t,j} \approx \epsilon^2 e_j^\top G_t e_j. \tag{15}$$

Thus action channels are only equally sensitive in the special case where the relevant diagonal and directional structure of G_t is uniform. In manipulation, this is unlikely: Cartesian motion, wrist orientation, and gripper timing couple differently to contact dynamics and task margins.

The same argument applies across time. For a fixed action perturbation direction v injected at two different rollout steps, the local risk is

$$r_t(v) \approx v^\top G_t v, \tag{16}$$

and G_t changes with B_t, $\Phi_{t\to T}$, and $\nabla h(s_T)$. Therefore early approach, contact, grasping, transfer, and placement windows can have different sensitivity even when the perturbation norm is unchanged.

Finally, for a module- or layer-level perturbation $\zeta_{i,t}$ whose first-order effect on the action is $\eta_t \approx J_{i,t}\zeta_{i,t}$, the corresponding risk is

$$r_{i,t} \approx \zeta_{i,t}^\top J_{i,t}^\top G_t J_{i,t} \zeta_{i,t}. \tag{17}$$

This shows why layer selection for quantization or compilation cannot be based only on parameter count, FLOPs, or latency. A module is attractive for acceleration only if it has high speed benefit and low sensitivity-weighted risk under $J_{i,t}^\top G_t J_{i,t}$. The practical consequence is the design rule we test later: not all action dimensions, rollout durations, or layer boundaries are equal.

Claim 4: offline drift is incomplete without state-distribution control. Let $\ell(s)$ denote a closed-loop-relevant loss, such as action drift, sensitivity-weighted action error, or risk of approaching a failure boundary. Offline validation estimates behavior under a fixed distribution, typically d_{π_θ} or a dataset distribution. Closed-loop accelerated execution instead samples from d_{π_q}:

$$\mathbb{E}_{s\sim d_{\pi_q}}[\ell(s)] = \mathbb{E}_{s\sim d_{\pi_\theta}}[\ell(s)] + \left(\mathbb{E}_{s\sim d_{\pi_q}}[\ell(s)] - \mathbb{E}_{s\sim d_{\pi_\theta}}[\ell(s)]\right). \tag{18}$$

Open-loop action drift controls only the fixed-distribution term. It does not by itself control the distribution-shift term induced by executing the perturbed policy. This term can matter because a small nominal action error can move the system away from the FP16 trajectory, after which the accelerated policy may visit states with larger implementation error, higher closed-loop sensitivity, or smaller task margins.

More formally, if ℓ is a nonnegative bounded loss with $0 \le \ell(s) \le \|\ell\|_\infty$, then the distribution-shift contribution obeys

$$\left|\mathbb{E}_{d_{\pi_q}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell]\right| \le \|\ell\|_\infty \,\mathrm{TV}(d_{\pi_q}, d_{\pi_\theta}), \tag{19}$$

under the standard definition $\mathrm{TV}(p,q) = \frac{1}{2}\|p-q\|_1$. If ℓ is L-Lipschitz under a state metric, an analogous Wasserstein bound gives

$$\left|\mathbb{E}_{d_{\pi_q}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell]\right| \le L\, W_1(d_{\pi_q}, d_{\pi_\theta}). \tag{20}$$

The point is not that these distances are easy to estimate in high-dimensional robot rollouts, but that fixed-distribution drift is only one term. Predicting rollout robustness also requires controlling how far the accelerated policy moves the visited state distribution.

Claim 5: aggregate success differences decompose exactly into repairs and regressions. For matched cases $c \in \mathcal{C}$ and binary success indicators $S_A(c), S_B(c) \in \{0,1\}$,

$$\sum_{c\in\mathcal{C}} S_B(c) - \sum_{c\in\mathcal{C}} S_A(c) = \sum_{c\in\mathcal{C}} \mathbf{1}\{S_A(c)=0, S_B(c)=1\} - \sum_{c\in\mathcal{C}} \mathbf{1}\{S_A(c)=1, S_B(c)=0\}. \tag{21}$$

This identity is simple but important: a net success gain can hide substantial behavioral churn. Paired repairs and regressions are therefore not an optional diagnostic; they are the decomposition of the aggregate number.

4 Experimental Setup

Model and task. We use the GR00T N1.5 LIBERO long-horizon checkpoint and evaluate on LIBERO-10 [9, 10]. The accepted FP16 baseline matches the official reference result of 38/50 on the standard 5-trial LIBERO-10 run. The evaluated policy predicts action chunks with seven action components: x, y, z, roll, pitch, yaw, and gripper. Unless noted otherwise, experiments use 8 denoising steps.

Two perturbation tracks. Track A is the quantization track. It applies selective W4A8 fake quantization and optional ATM/OHB compensation to the same FP16 policy, then evaluates both open-loop action drift and closed-loop rollout redistribution. Track B is the acceleration-boundary track. It keeps model weights in FP16, but changes the action-head execution boundary using `torch.compile` with optional eager islands around selected transformer blocks. Track B is included because deployment-oriented acceleration

can introduce small numerical or graph-boundary differences even when the model semantics are unchanged. We use it as a controlled stress test of the same closed-loop sensitivity claims, not as evidence of final packed-kernel quantization speed.

Quantization scope. We study the selective `llm_dit_mlp` W4A8 fake-quantization scope: 84 selected LLM linear layers and 32 selected DiT MLP linear layers, for 116 quantized modules. DiT attention projections are intentionally left in floating point. This design matters because attention projections feed a softmax routing mechanism, while MLP errors are more often residual perturbations.

Compensation modes. Following QuantVLA terminology, we evaluate two scale-calibrated compensation mechanisms: attention temperature matching (ATM) and output head balancing (OHB) [13]. Our implementation should be read as a reproduction-oriented approximation of these mechanisms rather than a claim of exact code-level equivalence to the original implementation. ATM rescales the DiT attention query to match teacher/student attention-logit standard deviation:

$$\alpha = \frac{\mathrm{std}_{\mathrm{teacher}}(\text{attention logits})}{\mathrm{std}_{\mathrm{student}}(\text{attention logits})}, \qquad \text{query} \leftarrow \alpha \cdot \text{query}. \tag{22}$$

OHB rescales the attention output before residual addition to match teacher/student attention-output RMS:

$$\beta = \frac{\mathrm{rms}_{\mathrm{teacher}}(\text{attention output})}{\mathrm{rms}_{\mathrm{student}}(\text{attention output})}, \qquad \text{attention output} \leftarrow \beta \cdot \text{attention output}. \tag{23}$$

The combined mode applies both corrections. In all cases the compensation acts on DiT attention processors; it does not mean DiT attention weights are quantized.

Offline drift protocol. The offline validation uses 16 real LIBERO calibration observations and 128 held-out real evaluation observations. Teacher and student calls use matched random seeds because action denoising begins from random Gaussian actions. We report NMSE, relative RMSE, cosine similarity, and max absolute difference.

Closed-loop protocol. The main closed-loop analysis evaluates LIBERO-10 initial states 0–14: 10 tasks, 15 initial states per task, and 150 episodes per policy/mode. For paired comparisons, each mode is evaluated on the same task-initialization pairs. We report Wilson 95% confidence intervals for aggregate success rates as a descriptive uncertainty estimate, but use paired repair/regression counts as the primary behavioral diagnostic.

5 Results

5.1 Offline Mean Drift Improves, But Individual Regressions Remain

On the 128 held-out real observations, ATM and OHB reduce mean action drift relative to uncompensated W4A8 (Table 1). The identity control exactly matches `none`, showing that the custom attention processor path itself does not introduce measurable drift.

Table 1: Offline teacher-student action drift on 128 held-out real LIBERO observations.

Config	Mode	Modules	NMSE mean	Rel. RMSE mean	Cosine mean
`llm_dit_mlp`	none	116	0.0178962	0.0981458	0.992492
`llm_dit_mlp`	identity	116	0.0178962	0.0981458	0.992492
`llm_dit_mlp`	ATM	116	0.0159471	0.0904183	0.993101
`llm_dit_mlp`	OHB	116	0.0160919	0.0853958	0.992769
`llm_dit_mlp`	ATM+OHB	116	0.0153168	0.0838784	0.993048

The mean improvement hides observation-level regressions. ATM+OHB has the best mean drift, while still worsening 34 of 128 held-out observations by NMSE. This is the first sign that mean offline drift is insufficient: compensation moves the action distribution closer on average, but not uniformly.

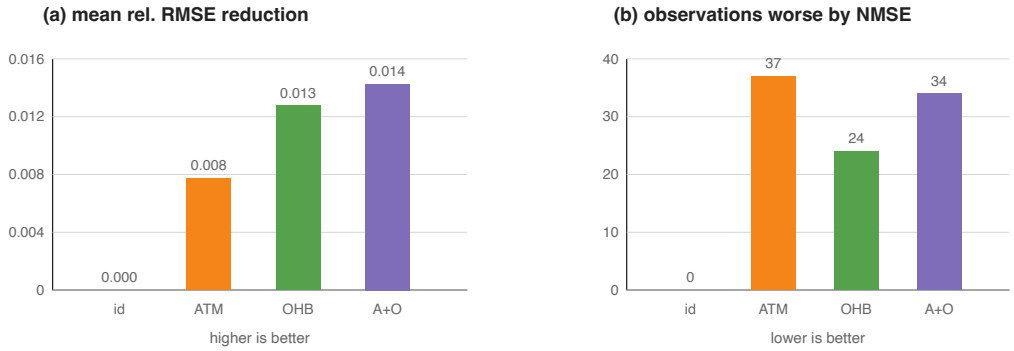

Figure 2: Offline mean drift versus observation-level regressions. Left: reduction in mean relative RMSE from uncompensated W4A8. Right: number of held-out observations whose NMSE worsens relative to uncompensated W4A8.

5.2 Selective W4A8 Remains in the FP16 Range

Over 150 LIBERO-10 task-initialization pairs, selective W4A8 remains in the same performance band as FP16 (Table 2). Some quantized variants have higher aggregate point estimates, but these gaps should not be read as evidence that quantization is inherently better than FP16. Rather, the quantized policies perturb the trajectory distribution. In finite closed-loop evaluation, that perturbation can push some failed FP16 trajectories into successful basins while pushing other successful trajectories into failure.

Table 2: Closed-loop LIBERO-10 success over task-initialization pairs 0–14.

Policy	Successes	Success rate (Wilson 95% CI)
FP16	108/150	72.0% [64.3, 78.6]
W4A8 `llm_dit_mlp` + none	113/150	75.3% [67.9, 81.5]
W4A8 `llm_dit_mlp` + ATM	114/150	76.0% [68.6, 82.1]
W4A8 `llm_dit_mlp` + OHB	116/150	77.3% [70.0, 83.3]
W4A8 `llm_dit_mlp` + ATM+OHB	114/150	76.0% [68.6, 82.1]

The confidence intervals are wide and overlapping, so the table should not be read as a statistical ranking of policies. The more stable conclusion is that selective W4A8 remains in the same behavioral range as FP16 under this protocol, while changing the identity of successful and failed rollouts.

5.3 Aggregate Rates Hide Task-Level Redistribution

The effects are task-dependent. ATM improves task 4 by 5 successes over `none`, but regresses task 8 by 5 successes. OHB is more balanced and gives the highest aggregate point estimate, but it still regresses tasks 3, 8, and 9 relative to `none`. The detailed task-level table appears in Appendix B.

5.4 Paired Rollout Flips Reveal The Mechanism

Paired comparisons over identical task-initialization pairs show that the compensation modes are not monotonic improvements (Table 3). OHB is not simply "3 episodes better" than `none`; it repairs 16 failures and introduces 13 new failures. Its net gain is small, but the underlying trajectory redistribution is substantial.

Table 3: Paired outcomes over the same 150 task-initialization pairs.

Comparison	Repaired	Regressed	Same success	Same failure	Net
ATM vs none	14	13	100	23	+1
OHB vs none	16	13	100	21	+3
OHB vs ATM	15	13	101	21	+2

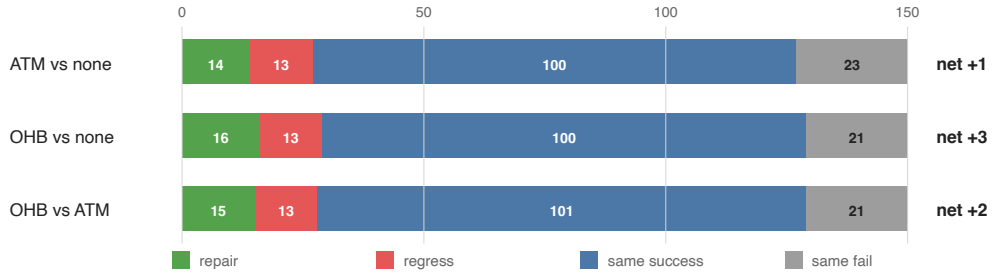

Figure 3: Paired closed-loop rollout flips. Net success-rate changes decompose into repaired failures, new regressions, unchanged successes, and unchanged failures.

For the disjoint init 5–14 comparison between FP16 and ATM+OHB W4A8, we observe the same pattern: 14 FP16 failures are repaired, while 8 FP16 successes become quantized failures. The aggregate gain of 6/100 comes from this difference.

5.5 Keyframe-Based Trajectory Diagnostics

To make the paired flips more concrete, we inspected contact sheets for representative repaired and regressed rollouts. Each contact sheet contains three matched policy rows—FP16, W4A8 `none`, and W4A8 ATM+OHB—sampled uniformly over the rollout. The sheets are included in the repository keyframe directory; they are diagnostic artifacts rather than a new quantitative benchmark.

Table 4: Representative keyframe diagnostics for paired rollout flips. Frame counts refer to the number of rendered rollout frames before success or horizon failure.

Case	Outcome pattern	Keyframe-level observation	Interpretation
Task 8 init 7	FP16 fail 991; ATM+OHB fail 991; `none` success 388	`none` enters a short successful approach branch early, while FP16 and ATM+OHB spend the late rollout in repeated correction.	Raw quantization can push a weak FP16 task slice out of a failure basin.
Task 4 init 10	FP16 fail 991; `none` fail 991; ATM+OHB success 242	ATM+OHB completes the mug/plate interaction quickly; the other rows remain in horizon-length correction.	Compensation can repair a task by changing contact timing or object-interaction order.
Task 0 init 3	FP16 success 250; ATM+OHB success 267; `none` fail 991	`none` disturbs an otherwise fast object-container relation and never recovers.	The same raw perturbation that repairs some cases can regress high-margin FP16 successes.
Task 6 init 1	FP16 success 222; `none` success 477; ATM+OHB fail 991	ATM+OHB remains visually stable but under-progresses around the decisive placement phase.	Balancing can be too conservative or shift progress timing, even when it reduces mean offline drift.
Task 8 init 0	FP16 success 657; ATM+OHB fail 991; `none` fail 991	Both quantized rows take the wrong side of a task-8 decision boundary that FP16 crosses successfully.	Task-level gains are not monotonic; the same task can contain repairs and regressions.

These examples support the trajectory-redistribution interpretation. The repaired cases do not look like small final-frame placement corrections; they usually branch earlier and terminate much sooner than the corresponding failures. The regressed cases show the complementary failure: a small policy perturbation can disrupt an early object relation or produce a stable but insufficiently progressive path. Thus the paired counts in Table 3 correspond to visible differences in trajectory basins, not only bookkeeping changes in aggregate success.

5.6 Sensitivity Maps: Not All Perturbations Are Equal

The preceding results show that quantization changes which rollouts succeed. This section is the bridge from that behavioral observation to an actionable design rule. We next ask a more diagnostic question: which perturbations are closed-loop sensitive? To isolate this from implementation details of a particular quantizer, we ran small controlled perturbation probes on two FP16-success cases: task 4 init 9 and task 6 init 8. These probes should not be read as a quantization method. They are local interventions that reveal which directions, rollout windows, and module boundaries are fragile enough that a future quantization or acceleration error could matter.

Table 5: Controlled action perturbations on two FP16-success cases. All full-horizon single-channel probes use the same continuous-action perturbation norm, $\|\delta a\|_2 = 0.03$. S/F denotes success/failure and the number of policy requests; horizon failures usually reach 991 requests.

Probe	Window	Successes	Per-case outcome
No perturbation	Full	2/2	Task 4 init 9: S224; task 6 init 8: S649
6D continuous	Full	0/2	F991; F991
x only	Full	1/2	S349; F991
y only	Full	0/2	F991; F991
z only	Full	2/2	S239; S666
Roll only	Full	0/2	F991; F991
Pitch only	Full	0/2	F991; F991
Yaw only	Full	0/2	F991; F991
y only, task 4	Early / middle / late	1/3	0:75 F991; 75:150 F991; 150:225 S219
y only, task 6	Early / middle / late	2/3	0:200 F991; 200:450 S697; 450:700 S752

Table 5 supports two practical claims. First, action dimensions are not equally sensitive. A z perturbation of the same norm is absorbed in both cases, while y and rotation perturbations are catastrophic in this small probe. Second, rollout time is not equally sensitive. The same y perturbation is damaging early, but late-window perturbations can be absorbed after the task has entered a more stable phase. This maps the first two axes of the sensitivity space: channel and time. It also matches the linearized view in Section 3: the relevant quantity is not only $\|\eta_t\|$, but also the direction of η_t and the future closed-loop gain multiplying it.

We also tested whether acceleration-oriented graph boundaries are behaviorally transparent. The following experiment compiles the action-head model while leaving either no block, block 0, or block 1 as an eager island. The variants have almost identical median server latency, but their closed-loop behavior differs sharply.

Table 6: Layer/boundary sensitivity under acceleration-oriented action-head execution. The same two task-initialization pairs are used as in Table 5.

Execution mode	Successes	Task 4 init 9	Task 6 init 8	p50 latency	Speedup
FP16 baseline	2/2	S224	S649	160.53 ms	1.00×
Compile `action_head_model`	2/2	S222	S916	72.80 ms	2.21×
Compile + block 0 eager	1/2	F991	S404	72.62 ms	2.21×
Compile + block 1 eager	0/2	F991	F991	73.10 ms	2.20×

The full compiled action head is fast and succeeds on both cases, but it is not exactly behavior-transparent: task 6 takes 916 requests instead of 649. More importantly, making a small eager island does not monotonically improve safety. Block 0 eager fails task 4 but succeeds task 6 faster than the full compiled variant, while block 1 eager fails both. Because all three accelerated variants have nearly the same p50 latency, the result

cannot be explained by speed alone. It is a layer/boundary sensitivity effect: changing where numerical or graph-boundary differences enter the policy can move a rollout into a different basin.

Together, the channel, time, and layer-boundary probes turn the earlier repair/regression phenomenon into a sensitivity map. These two FP16-success cases do not define a global ranking. Rather, they show that closed-loop risk is structured enough to guide what should be quantized, compiled, protected, or left in higher precision.

Finally, we compared closed-loop traces to locate the first divergence. Table 7 summarizes direct comparisons from the accelerated runs. The main pattern is temporal separation: action differences appear first, millimeter-scale state differences appear later, and outcome flips occur much later through accumulated contact and progress changes.

Table 7: First-divergence diagnostics for accelerated variants. Position thresholds are measured between matched simulator states; action thresholds use continuous-action coordinates.

Case	Pair	Outcome	First action > 0.01	First state > 1 mm	Gripper mismatch
Task 4 init 9	Full → blk0-eager	S222 → F990	Step 21	Step 59, 1.186 mm, z	Step 99
Task 4 init 9	Full → blk1-eager	S222 → F990	Step 8	Step 59, 1.012 mm, x	Step 97
Task 6 init 8	Full → blk0-eager	S916 → S404	Step 8	Step 58, 1.041 mm, x	Step 327
Task 6 init 8	Full → blk1-eager	S916 → F990	Step 53	Step 58, 1.133 mm, y	Step 261

These first-divergence traces explain why latency-equivalent implementations can have different closed-loop outcomes. The decisive failure is rarely a single large action jump. Instead, small early action differences create a delayed state separation; depending on the task margin and contact phase, that separation is either absorbed, repaired into a shorter trajectory, or amplified into horizon failure. This supports a stronger diagnostic principle: for VLA quantization and acceleration, one should map sensitivity across action channels, rollout phases, and module boundaries before choosing what to quantize, compile, or protect.

Using sensitivity proxies to guide protection. The previous probes identify sensitive boundaries; the next question is whether this information can guide an engineering choice. We therefore ran a small proxy-guided mixed-precision execution experiment on 15 matched LIBERO task-initialization pairs drawn from the fragile task slices. This is not a final deployment benchmark. It is a diagnostic test of whether a sensitivity proxy can improve the trade-off between warm serving latency and closed-loop success.

Table 8: Proxy-guided mixed-precision execution on 15 matched task-initialization pairs. "Repair/regress" is measured relative to the speed-only compiled action-head variant.

Mode	Execution policy	Success	Server p50	Speedup	Repair/regress vs speed-only
FP16 baseline	No compile	7/15	84.76 ms	1.00×	–
Speed-only	Compile `action_head.model`	5/15	50.35 ms	1.68×	–
Proxy block 0	Compile action head; keep block 0 eager	6/15	50.96 ms	1.66×	3 / 2
Proxy blocks 8–15	Compile action head; keep blocks 8–15 eager	7/15	67.36 ms	1.26×	3 / 1
Random block 1	Compile action head; keep block 1 eager	5/15	51.54 ms	1.64×	2 / 2

Table 8 shows the trade-off directly. Speed-only compilation is fastest, but drops from 7/15 to 5/15 successes, with three baseline successes becoming failures. Protecting blocks 8–15, selected from closed-loop repair/regression evidence, restores the aggregate count to 7/15 while retaining a 1.26× median server-latency speedup. It repairs three speed-only failures (task 4 init 6, task 6 init 0, task 8 init 10) and introduces one new regression (task 4 init 9). A random block-1 eager island has similar latency to speed-only but remains at 5/15, suggesting that protection location matters.

The result should not be read as eliminating closed-loop perturbation. Relative to FP16, the proxy-guided variant still redistributes outcomes: it repairs task 8 init 9 but regresses task 4 init 9. The practical conclusion is narrower and more useful: a closed-loop sensitivity proxy can reduce the damage caused by speed-only acceleration, but paired repair/regression analysis is still needed because the protected policy is not behaviorally identical to the baseline.

Table 9: Selected first-divergence diagnostics for the speed-only and proxy-guided blocks 8–15 variants. The action threshold is $\ell_2 > 0.05$ in continuous action coordinates; EEF thresholds are Euclidean position differences.

Case	Speed-only	Proxy	First action	First EEF > 1 cm	First EEF > 5 cm	Interpretation
4:6	F990	S241	35	83	192	Proxy restores a short successful branch close to baseline.
6:0	F990	S206	62	120	143	Proxy restores the baseline success basin.
8:10	F990	S604	87	141	467	Proxy repairs outcome through a longer branch.
8:9	S476	S424	55	157	166	Proxy preserves a beneficial speed-only branch.
4:9	S222	F990	57	104	125	Proxy creates a new regression.

Table 9 explains why the guide can help without being exact. In the repaired cases, action differences appear tens of steps before centimeter-scale EEF divergence, and outcome flips occur much later. The proxy-guided policy sometimes returns to a baseline-like basin, as in task 4 init 6 and task 6 init 0, but sometimes enters a different successful branch, as in task 8 init 10. The new task 4 init 9 regression is the complementary failure mode: protection changes the graph-boundary perturbation enough to cross a different closed-loop basin boundary.

5.7 Empirical Support For The Theoretical Claims

The theoretical claims in Section 3 are not used as proof that one accelerated implementation is superior. They organize what must be measured. The experiments support the claims in the following sense.

Table 10: How the empirical results instantiate the perturbation claims.

Claim	Experimental evidence	Interpretation
Claim 1: sensitivity propagation	Small action changes produce both repairs and regressions under the same model family. Keyframe repairs branch early rather than only correcting final placement.	The effect depends on where the perturbation enters the trajectory, not only on its norm.
Claim 2: margin crossing	Task 8 contains both directions: W4A8 `none` improves task 8 from 3/15 FP16 successes to 9/15, yet task 8 init 0 is a case where both quantized modes regress a successful FP16 rollout.	The same task can contain low-margin failures that are repaired and low-margin successes that are broken.
Claim 3: anisotropic sensitivity	Equal-norm action probes differ by channel and phase; acceleration variants with similar p50 latency differ from 2/2 success to 0/2 depending on layer boundary. In a 15-case proxy-guided probe, protecting blocks 8–15 restores speed-only from 5/15 to 7/15 successes, while random block 1 remains at 5/15.	Perturbation risk is structured by action direction, future closed-loop gain, task margin, and module location. Sensitivity-guided protection can improve the speed–robustness trade-off, but does not eliminate trajectory redistribution.
Claim 4: offline drift insufficiency	ATM+OHB has the best mean offline drift, but OHB has the highest aggregate closed-loop point estimate; ATM improves task 4 by 5 successes over `none` while regressing task 8 by 5.	Reducing average open-loop error does not imply monotonic closed-loop improvement.
Claim 5: repair/regression decomposition	OHB vs `none` has 16 repaired failures and 13 new regressions, yielding only a net +3 successes. ATM vs `none` is 14 repairs and 13 regressions.	Aggregate success changes hide substantial redistribution of which rollouts succeed.

6 Analysis

Why can a quantized policy improve a rollout? The quantized policy is not guaranteed to be worse than FP16 in a finite simulator benchmark. FP16 is not an oracle; it is a learned policy with its own fragile regions. A small action perturbation can move a trajectory away from a failure mode. This is especially plausible when the FP16 policy is already weak on a task slice, such as LIBERO task 8 in our baseline. Therefore, an aggregate improvement should not be read as "low precision is better." It means the perturbed policy enters different trajectory basins, some of which are more favorable under the benchmark's finite initial states and success predicates.

Why does offline drift fail to predict closed-loop behavior? Offline drift is measured on a fixed observation set. Closed-loop rollouts produce observations adaptively. Once a quantized action changes the environment state, later observations may differ from the offline distribution. This creates two mismatches: the local action error estimate may not apply to the new states, and the success function may be discontinuous around contact and placement boundaries. This explains why ATM+OHB can have the best mean offline NMSE but not the best closed-loop aggregate success.

A control-theoretic interpretation. From a feedback-control perspective, post-training acceleration acts like a structured input disturbance injected into the learned controller. For the quantization track, write

$$a_t^q = \pi_\theta(s_t^q, l) + \eta(s_t^q, l), \tag{24}$$

where η is the action perturbation induced by quantization and compensation. For a compile or kernel-replacement track, the same form applies with η denoting the action difference induced by the accelerated execution path. Linearizing around an FP16 trajectory gives

$$\delta s_{t+1} \approx (A_t + B_t K_t)\delta s_t + B_t \eta_t, \tag{25}$$

where $A_t = \partial f/\partial s$, $B_t = \partial f/\partial a$, and $K_t = \partial \pi_\theta/\partial s$ along the trajectory. The closed-loop sensitivity is therefore governed not only by the magnitude of η_t, but also by the local closed-loop gain induced by the environment dynamics and the policy feedback. A small action perturbation can be attenuated in low-gain regions and amplified near contact, grasping, or placement transitions.

This also makes success a margin problem. Let $h(s_T)$ denote an implicit terminal success margin, with success when $h(s_T) > 0$. A rollout with large positive margin can tolerate sizeable action perturbations, while a low-margin rollout can flip if acceleration moves $h(s_T)$ across zero. This explains why paired outcomes contain both repairs and regressions: an accelerated implementation can move a trajectory away from one failure basin while pushing another trajectory across a success boundary. The useful diagnostic is therefore not just average action error, but the alignment between action perturbations, closed-loop sensitivity, and task success margins.

Why is selective W4A8 robust? The `llm_dit_mlp` scope leaves DiT attention projections in floating point. This avoids quantizing the most routing-sensitive part of the action head. A perturbation to $QK^\top$ before softmax can change attention entropy and token routing. By contrast, MLP quantization more often enters as an additive residual perturbation,

$$y_{\text{student}} = y_{\text{teacher}} + \epsilon. \tag{26}$$

Layer normalization, residual connections, diffusion denoising, and receding-horizon replanning can absorb some of this error. This helps explain why uncompensated W4A8 already reaches 113/150 successes.

Why are ATM and OHB not additive? ATM changes where attention looks by changing the attention-logit scale. OHB changes how strongly the attention output enters the residual stream. If ATM moves routing into a different pattern, OHB rescales a different output than the one calibrated under the uncompensated student. The combined correction can overcorrect, undercorrect, or move the policy into a different trajectory basin. This is consistent with ATM+OHB reaching 114/150, below standalone OHB at 116/150.

What should be reported for VLA acceleration? The minimum evidence package should include offline action drift on held-out real observations, per-component action error, closed-loop success on matched task-initialization pairs, per-task and per-init success rates, paired repair/regression counts, latency and memory under the same protocol, and an explicit distinction between fake quantization, prototype compile paths, and final packed-kernel deployment.

7 Implications

Evaluation. VLA inference acceleration should be evaluated as a closed-loop policy perturbation. Reporting only MSE, aggregate success, or latency can be misleading. Paired rollout flips are a compact way to expose whether a faster implementation is a monotonic improvement or a redistribution of successes.

Calibration and protection. Calibration or acceleration planning should not only minimize average layerwise error or maximize local speed. It should consider which errors matter for downstream control. A useful calibration or profiling set should cover fragile contact states, low-margin success boundaries, and task slices where the FP16 policy is already weak. A control-aware objective would approximate Equation 13: weight errors by estimated closed-loop sensitivity, not only by their open-loop magnitude. This points toward rollout-informed calibration sets, residual action correction, sensitivity-guided compile boundaries, or risk-gated precision fallback as natural next steps beyond ATM/OHB-style statistic matching.

Closed-loop credit assignment. This perspective borrows an idea from reinforcement learning without requiring full policy optimization: local action errors should be assigned credit according to their downstream effect on future rollout outcomes. In standard PTQ or speed-only compilation, an error η_t is usually priced by its immediate norm or ignored if latency improves. In the closed-loop view, its risk is closer to $\eta_t^\top G_t \eta_t$, which assigns larger cost to directions that propagate through dynamics and reduce terminal task margin. This suggests a lightweight alternative to direct RL: use paired rollouts, first-divergence states, or low-margin states to estimate sensitivity proxies, then guide calibration, mixed precision, residual correction, compile boundaries, kernel replacement, or fallback decisions. The proxy-guided mixed-precision probe in Table 8 is an initial example: a closed-loop proxy improves the repair/regression balance relative to speed-only execution, while a random protection island does not.

Sensitivity-guided acceleration. For VLA or world-model deployment, layer selection should not be based only on parameter count, FLOPs, compiler convenience, or latency. A useful acceleration candidate should have both high compute benefit and low closed-loop sensitivity. Informally, for a module i with expected speed benefit g_i and acceleration-induced perturbation η_i, the relevant risk is closer to a sensitivity-weighted quantity,

$$r_i \approx \mathbb{E}\left[\eta_i^\top G_i \eta_i\right], \tag{27}$$

where G_i summarizes downstream dynamics, feedback, and task-margin sensitivity. This suggests a mixed-precision and mixed-execution selection rule: prioritize modules with large g_i/r_i, and leave routing-sensitive, contact-critical, or margin-critical components in higher precision or eager execution unless closed-loop tests show otherwise.

The small acceleration-boundary experiment illustrates both sides of this rule. Compiling the entire action head maximizes warm serving speed but increases closed-loop regressions. Protecting blocks 8–15 gives up part of that speed but restores the baseline aggregate success count on the matched 15-case slice. This is not a guarantee of safety: it still repairs one FP16 failure and regresses one FP16 success. The deployment lesson is therefore to treat mixed precision as a constrained optimization over speed benefit and closed-loop risk, not as a one-shot numerical equivalence transform.

Fine-tuning. Acceleration-aware fine-tuning is policy adaptation under deployment constraints, including low precision, compiled execution, or custom kernels. If the objective is only behavior cloning on static observations, it may still miss closed-loop state distribution shift. More policy-aware objectives may be needed, including rollout-informed calibration, DAgger-style data aggregation, or task-slice reweighting.

Deployment claims. Our results are behavior-level evidence for two prototype acceleration paths: fake W4A8 quantization and action-head compilation boundaries. They do not establish final low-precision latency, memory, or energy improvements. A complete deployment claim requires packed low-precision kernels or production compile/graph backends, plus end-to-end inference profiling under the same closed-loop protocol.

8 Limitations

This study has several limitations. The quantized implementation uses fake quantization, not a final packed int4/int8 deployment backend. The mixed-precision acceleration probe uses prototype action-head compilation boundaries and only 15 matched task-initialization pairs; it should be read as diagnostic evidence for the guide rather than a deployment benchmark. Results are from one GR00T N1.5 checkpoint and LIBERO-10, not multiple VLA families. The main closed-loop benchmark has 150 task-initialization pairs; larger runs would provide tighter statistical confidence, and the current aggregate success intervals overlap. We do not run real-robot experiments. ATM/OHB are implemented as reproduction-oriented calibration mechanisms; exact implementation details may differ from the original paper. The keyframe diagnostics cover representative flips, but they are qualitative and not an exhaustive geometric measurement for every paired outcome.

These limitations do not weaken the central methodological point: local action approximation and closed-loop policy behavior are distinct evaluation targets.

9 Conclusion

Post-training inference acceleration of VLA policies should be viewed as policy perturbation. Even small action errors can matter because they enter a feedback loop, alter the closed-loop state distribution, and interact with task-dependent success margins. In our GR00T/LIBERO reproduction, selective W4A8 quantization remains behaviorally robust and can shift aggregate point estimates, but those shifts come from task-dependent repair/regression redistribution rather than uniform dominance. Prototype action-head compilation gives substantial warm serving speedups, yet speed-only execution can introduce closed-loop regressions. Sensitivity-guided protection improves the acceleration–robustness trade-off in our small probe, but it reduces rather than eliminates closed-loop trajectory redistribution.

The main lesson is practical: VLA acceleration papers should report paired closed-loop rollouts, not only offline drift, aggregate success, or latency. Small action errors matter when they change the trajectory basin.

A Keyframe Contact Sheets

The following contact sheets support the qualitative diagnostics in Table 4. They are enlarged here for visual inspection. Each sheet shows matched FP16, W4A8 `none`, and W4A8 ATM+OHB rows sampled uniformly over the rollout.

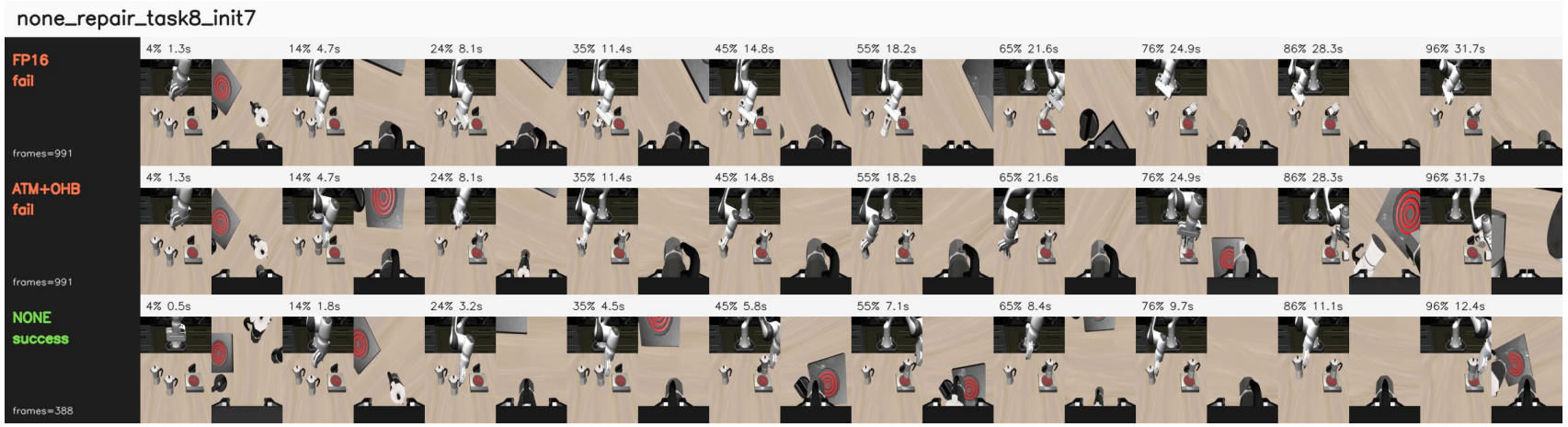

Figure 4: Task 8 init 7: W4A8 `none` repairs FP16 and ATM+OHB horizon failures by entering a shorter successful branch.

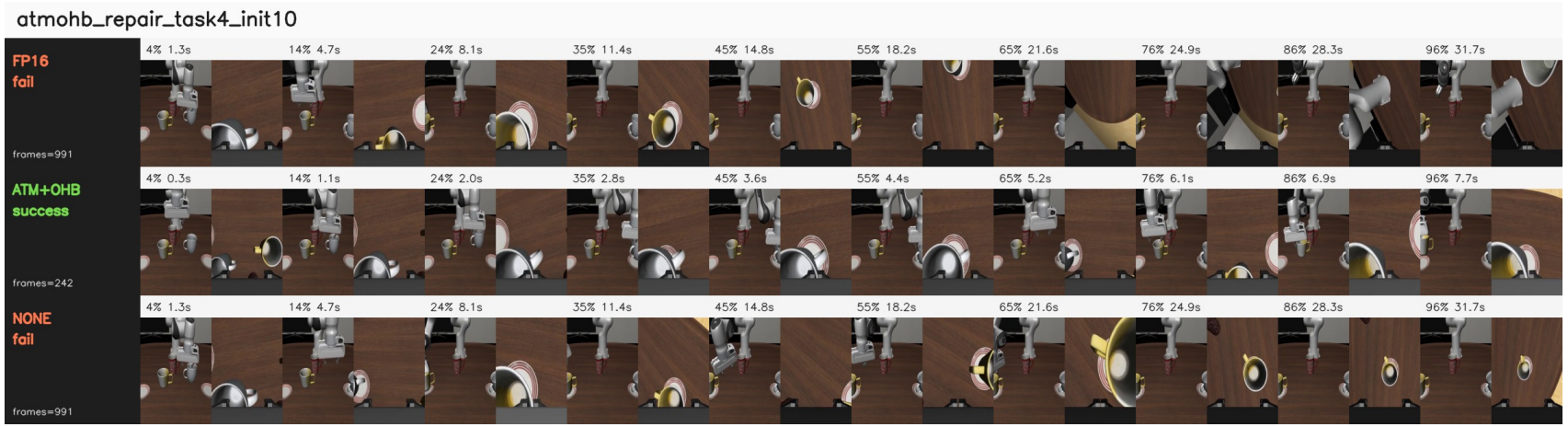

Figure 5: Task 4 init 10: ATM+OHB repairs both FP16 and W4A8 `none` failures with an early mug/plate completion.

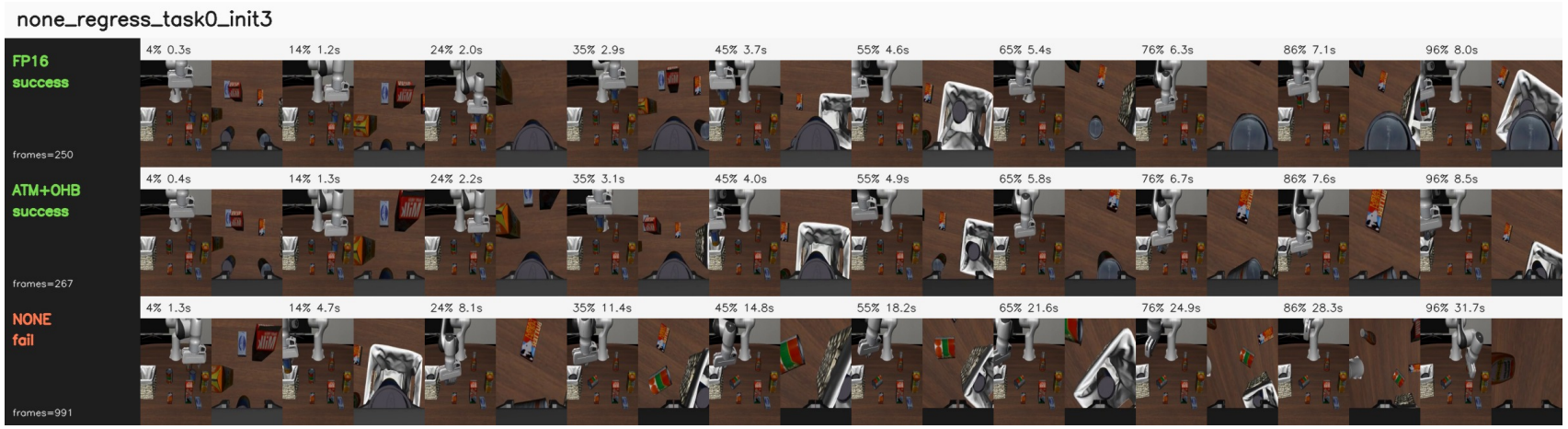

Figure 6: Task 0 init 3: W4A8 `none` regresses a fast FP16/ATM+OHB success by disturbing the early object-container relation.

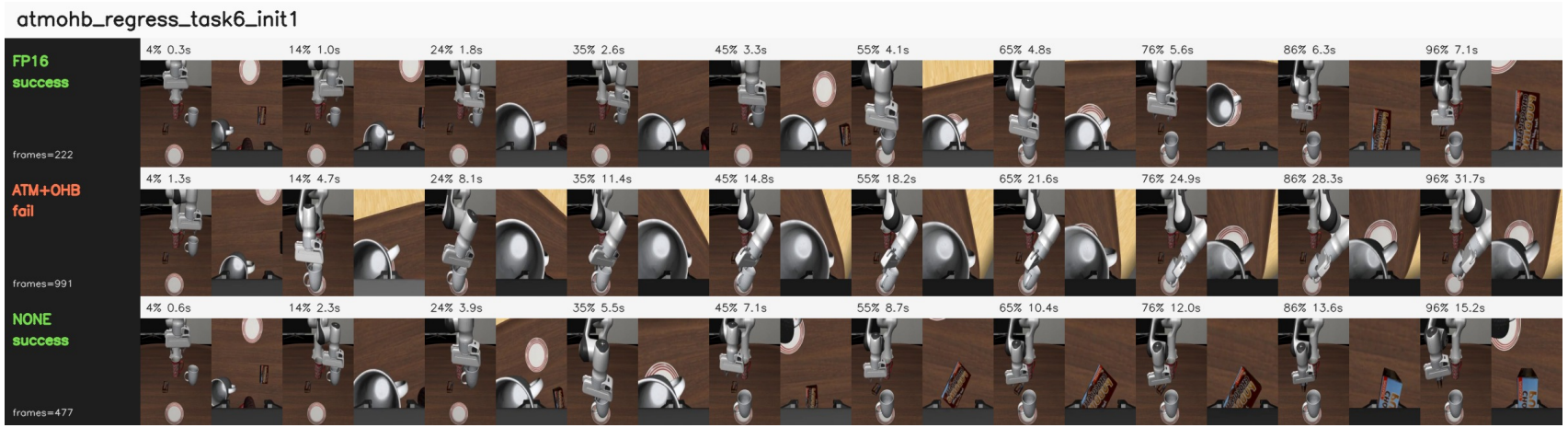

Figure 7: Task 6 init 1: ATM+OHB remains visually stable but under-progresses around the decisive placement phase, while FP16 and W4A8 `none` succeed.

Figure 8: Task 8 init 0: both quantized modes regress a successful FP16 rollout, showing that task 8 contains both quantization repairs and quantization regressions.

B Detailed Experimental Tables

B.1 Standard 5-Trial FP16 and ATM+OHB W4A8

Table 11: Standard 5-trial LIBERO-10 comparison.

Task id	FP16	W4A8 ATM+OHB	Delta
0	5/5	5/5	0
1	3/5	4/5	+1
2	3/5	5/5	+2
3	5/5	5/5	0
4	4/5	4/5	0
5	5/5	5/5	0
6	3/5	2/5	-1
7	4/5	3/5	-1
8	1/5	0/5	-1
9	5/5	5/5	0
Total	38/50	38/50	0

B.2 Disjoint Init 5–14 FP16 and ATM+OHB W4A8

Table 12: Disjoint init 5–14 comparison.

Task id	FP16	W4A8 ATM+OHB	Delta
0	8/10	8/10	0
1	8/10	10/10	+2
2	9/10	8/10	-1
3	10/10	10/10	0
4	4/10	8/10	+4
5	10/10	10/10	0
6	3/10	5/10	+2
7	7/10	7/10	0
8	2/10	3/10	+1
9	9/10	7/10	-2
Total	70/100	76/100	+6

B.3 Ablation: W4A8 None, ATM, OHB

Table 13: Ablation over init 0–14.

Task id	None	ATM	OHB	ATM-None	OHB-None
0	10/15	11/15	13/15	+1	+3
1	13/15	13/15	14/15	0	+1
2	13/15	14/15	15/15	+1	+2
3	15/15	13/15	13/15	-2	-2
4	8/15	13/15	12/15	+5	+4
5	14/15	14/15	15/15	0	+1
6	9/15	10/15	8/15	+1	-1
7	8/15	8/15	8/15	0	0
8	9/15	4/15	6/15	-5	-3
9	14/15	14/15	12/15	0	-2
Total	113/150	114/150	116/150	+1	+3

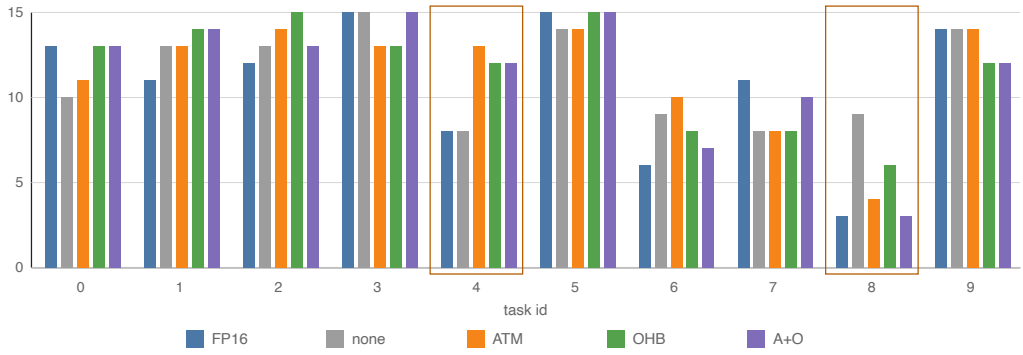

Figure 9: Task-wise success redistribution across policy variants. Aggregate changes arise from heterogeneous task-level gains and regressions rather than uniform improvement.

References

[1] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL `https://fbsbook.org`.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. URL `https://arxiv.org/abs/2212.06817`.

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL `https://arxiv.org/abs/2307.15818`.

[4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL `https://arxiv.org/abs/2303.04137`.

[5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022. URL https://arxiv.org/abs/2208.07339.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL https://arxiv.org/abs/2210.17323.

[7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. URL https://arxiv.org/abs/2406.09246.

[8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL https://arxiv.org/abs/2306.00978.

[9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL https://arxiv.org/abs/2306.03310.

[10] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL https://arxiv.org/abs/2503.14734.

[11] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *arXiv preprint arXiv:1011.0686*, 2010. URL https://arxiv.org/abs/1011.0686.

[12] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. URL https://arxiv.org/abs/2211.10438.

[13] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026. doi: 10.48550/arXiv.2602.20309. URL https://arxiv.org/abs/2602.20309.